

1. Objetivo del Proyecto

El propósito de este trabajo es demostrar, de manera práctica y fundamentada, la aplicación de técnicas de machine learning vistas a lo largo del curso para la resolución de un problema real de predicción de precios. Como consultor de ML, se espera:

1. **Seleccionar** uno de los algoritmos de regresión presentados en el curso, argumentando la elección con evidencias cuantitativas (comparación de métricas en validación cruzada).
2. **Construir** un modelo de regresión que estime el precio total (realSum) de una reserva, a partir de las variables descriptivas del inmueble y del booking.
3. **Evaluar** de forma rigurosa la precisión del modelo usando **Mean Absolute Error (MAE)**, y comparar su desempeño frente a una línea base simple.
4. **Analizar** el impacto de las etapas de preprocesamiento (tratamiento de nulos, escalado, encoding de variables categóricas, etc.) sobre los resultados.
5. **Discutir** las ventajas y desventajas del modelo implementado (interpretabilidad, capacidad de generalización, tiempo de entrenamiento, sensibilidad a hiperparámetros).
6. **Concluir** con recomendaciones prácticas para su despliegue en un entorno de producción o para la mejora de futuras versiones.

2. Descripción del Dataset

El conjunto de datos consta de dos archivos principales:

- **Train.csv** (36 195 filas, IDs 1–36 195): datos históricos utilizados para el entrenamiento y validación interna del modelo.
- **Test.csv** (15 513 filas, IDs 36 196–51 708): registros reservados para generar las predicciones finales y preparar la submission en Kaggle.

2.1 Variables de Entrada

Variable	Descripción
room_type	Tipo de propiedad: puede ser “private”, “shared” o “entire home/apt”.
room_shared	Indicador binario: la propiedad es compartida.
room_private	Indicador binario: la propiedad es privada (no compartida).
person_capacity	Máximo número de personas permitidas en la propiedad.
host_is_superhost	Indicador binario: el anfitrión es superhost.
multi	Indicador binario: la propiedad tiene múltiples habitaciones.
biz	Indicador binario: reserva de uso business o uso familiar.
cleanliness_rating	Calificación de limpieza de la propiedad (escala numérica).
guest_satisfaction_overall	Calificación global de satisfacción de los huéspedes (escala numérica).
bedrooms	Número de habitaciones del inmueble.
dist	Distancia al centro de la ciudad (en kilómetros).
metro_dist	Distancia a la estación de metro más cercana (en kilómetros).
lng, lat	Coordenadas geográficas (longitud y latitud) de la unidad de alojamiento.
city	Ciudad donde se ubica la propiedad.
day_type	Día de la semana en que se realizó la reserva (p. ej., “weekday”, “weekend”, etc.).
Target	
realSum	Precio total del booking (variable objetivo a predecir, en la moneda correspondiente).

2.2 Consideraciones Iniciales

- Naturaleza de las variables:

- Las variables como `room_type` y `day_type` requieren codificación (one-hot encoding o label encoding).
- Indicadores binarios (`room_shared`, `host_is_superhost`, etc.) se mantendrán tal cual o se convertirán a 0/1 según convenga al algoritmo.
- **Distribución del target:**
 - Antes de modelar, se explorará la distribución de `realSum` para detectar sesgos, valores atípicos o transformaciones necesarias (p. ej., logaritmo).
- **Datos geoespaciales:**
 - Las coordenadas (`lng`, `lat`) pueden contribuir a capturar diferencias de precio por ubicación; se evaluará su efecto o su agrupación por clusters de proximidad.
- **Partición de datos:**
 - Se reservará un subconjunto del `Train.csv` para validación (por ejemplo, validación cruzada k-fold) y se probará el modelo final en `Test.csv` para generar la entrega.

2. Carga y Validación de Datos

En esta sección se describen los pasos iniciales para preparar el conjunto de datos antes del análisis exploratorio. Incluye la importación de archivos, verificación de tipos y rangos, detección de valores faltantes y duplicados, así como la consistencia de los identificadores.

2.1 Importación de Archivos CSV

Se cargan los dos archivos principales del proyecto:

- **Train.csv** (36 195 filas) para entrenamiento y validación interna.
- **Test.csv** (15 513 filas) para generar la submission final en Kaggle.

Los datos se leen con `pd.read_csv()`, asegurando que las rutas apunten correctamente a los archivos.

2.2 Verificación de Estructuras y Ranges

- **Tipos de datos (dtypes):** Se imprimen los tipos de cada columna para confirmar que las variables numéricas, categóricas y booleanas estén correctamente representadas.
- **Valores numéricos (describe):** Se extraen estadísticas descriptivas (media, desviación, mínimos, máximos) tanto de train como de test para revisar rangos plausibles y detectar posibles entradas erróneas (por ejemplo, distancias negativas o precios atípicos).

2.3 Detección de Valores Faltantes

Se computa la cantidad de nulos por columna en ambos datasets:

- Variables críticas como realSum no deben contener nulos.
- Otras columnas numéricas o flags booleanos con valores faltantes serán imputadas en pasos posteriores.

2.4 Filas Duplicadas

Antes de eliminar duplicados, se cuenta cuántas filas repetidas existen en cada conjunto. Tras aplicar `drop_duplicates()`, se verifica que queden cero casos duplicados, garantizando la integridad de los registros.

2.5 Consistencia de Identificadores

- Se comprueba el rango de la columna id en train y test, asegurando que no existan solapamientos y que cubran los intervalos esperados (1–36 195 para train; 36 196–51 708 para test).
- Se verifica que el número de IDs únicos coincida con el número de filas, de modo que no falten ni sobren registros.

2.6 Preprocesamiento Numérico Previo al EDA

Antes de realizar el análisis exploratorio de datos, se ejecuta un preprocesamiento ligero para normalizar banderas booleanas y garantizar que todas las columnas numéricas estén en formato numérico:

1. Unificación de Flags Booleanos:

- Columnas como `room_shared`, `host_is_superhost`, `multi`, etc., se convierten a 0/1 mapeando valores textuales (TRUE/FALSE) y numéricos ('1'/'0'). Cualquier valor inválido se asigna a 0.

2. Conversión de Columnas Numéricas:

- Se eliminan comas de miles en cadenas y se convierten a valores numéricos con `pd.to_numeric(errors='coerce')`.
- Se cuenta cuántos NaN aparecieron tras la conversión en cada columna.

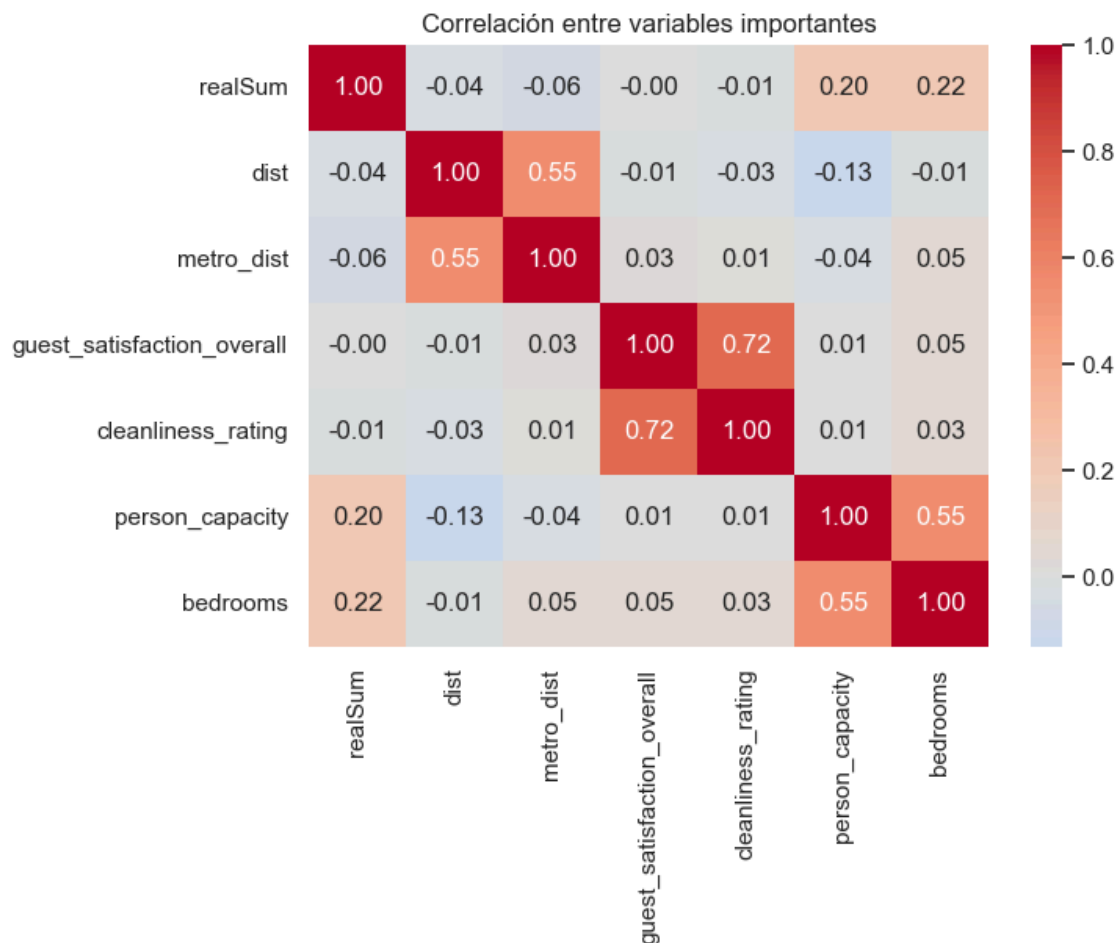
3. Tratamiento de Precios Inválidos:

- Se descartan todas las filas sin un valor válido de `realSum`, ya que es la variable objetivo.

4. Imputación de Otras Columnas Numéricas:

- Se aplica un `SimpleImputer(strategy='median')` a las columnas numéricas para reemplazar los posibles NaN restantes con la mediana respectiva.

Con estos pasos, el DataFrame de entrenamiento (`train`) queda libre de duplicados, valores faltantes en la variable objetivo y con las columnas numéricas correctamente tipadas. A partir de aquí, los datos están listos para el Análisis Exploratorio de Datos (EDA), donde se profundizará en distribuciones, correlaciones y detección de outliers.



3. Ingeniería de Características

En esta etapa se crean nuevas variables a partir de las existentes con el fin de enriquecer el espacio de entrada y mejorar la capacidad predictiva del modelo. La ingeniería de características consiste en aplicar conocimiento del dominio y técnicas matemáticas para derivar atributos más representativos que capturen relaciones útiles en los datos.

3.1 Combinación de Distancias

Al sumar la distancia al centro de la ciudad (dist) y la distancia a la estación de metro (metro_dist), surge la variable **total_dist**, que mide la accesibilidad global de la propiedad. Este agregado refleja de forma más completa el “coste de traslado” para el huésped, integrando ambos factores de proximidad en un único indicador .

3.2 Transformación Logarítmica para Exploración

La transformación **log1p** aplicada al precio (**realSum**) genera **log_realSum**, reduciendo la asimetría de la distribución original y atenuando la influencia de valores atípicos extremos. Esto facilita el análisis exploratorio y, en caso de emplear modelos lineales, puede mejorar su ajuste al aproximar la distribución de la variable objetivo a una forma más normalizada.

3.3 Interacción “Capacidad × Dormitorios”

La característica **cap_bedrooms**, producto de la capacidad máxima de personas y el número de dormitorios, captura la densidad de ocupación de la propiedad. Un mayor cociente de personas por habitación puede influir en la percepción de confort y, en consecuencia, en el precio, aportando una dimensión adicional al análisis.

3.4 Persistencia de los Datos Procesados

Guardar los conjuntos de datos con las nuevas variables en archivos separados es una práctica clave para la reproducibilidad y trazabilidad del flujo de trabajo. Mantener un repositorio de versiones de los datos procesados permite regenerar experimentos y auditar transformaciones, alineándose con principios FAIR de datos (encontrable, accesible, interoperable y reusable) y herramientas de control de versiones para datos como DVC.

Con estas operaciones de **combinación**, **transformación** e **interacción**, se enriquecen las fuentes de información disponibles para el modelo, lo que, en la mayoría de los casos, se traduce en un mejor desempeño predictivo y una mayor interpretación de los factores que afectan el precio de las reservas.

4. Selección de Modelo

En esta etapa se compararon diversas técnicas de regresión para determinar cuál ofrecía el mejor desempeño predictivo, medido mediante el Error Absoluto Medio (MAE) en validación cruzada de 5 pliegues. El proceso siguió estos pasos:

1. Definición del procedimiento

- Se empleó validación cruzada con 5 particiones aleatorizadas, garantizando que cada muestra de entrenamiento y prueba se evaluara de manera independiente.
- El criterio de evaluación fue el MAE, ya que refleja de manera directa la desviación promedio en unidades monetarias del precio predicho respecto al precio real.

2. Modelos Evaluados

Se probaron diez algoritmos representativos de diferentes familias:

- **Lineales:** Regresión Lineal, Ridge, Lasso

- **Árboles y Ensamblados:** Árbol de Decisión, Random Forest, Extra Trees, Gradient Boosting
- **Boosting Especializado:** XGBoost, LightGBM
- **Otros:** Support Vector Regressor (SVR), K-Nearest Neighbors (no incluido en el resumen final por su desempeño inferior)

3. Resultados de MAE Promedio

A continuación se presentan, ordenados de mejor a peor MAE medio en validación cruzada:

- **Random Forest:** 68.29 ± 2.48
- **Extra Trees:** 69.67 ± 2.07
- **Árbol de Decisión:** 76.47 ± 2.80
- **XGBoost:** 78.80 ± 2.14
- **LightGBM:** 82.97 ± 2.02
- **Gradient Boosting (sklearn):** 84.63 ± 2.22
- **SVR:** 97.69 ± 2.16
- **Lasso:** 101.30 ± 2.11
- **Ridge:** 102.83 ± 2.05
- **Regresión Lineal:** 102.84 ± 2.05

4. Análisis de los Resultados

- **Ensamble de Bosques (Random Forest y Extra Trees)**
Los métodos basados en ensambles de árboles se destacaron claramente, con Random Forest obteniendo el MAE más bajo. Esto se atribuye a su robustez frente a outliers y variables correlacionadas, así como a su capacidad de modelar relaciones no lineales complejas sin requerir ingente ajuste de parámetros.
- **Boosting Especializado (XGBoost, LightGBM)**
Aunque XGBoost mostró un desempeño respetable ($MAE \approx 78.8$), no superó al ensamble de bosques. LightGBM y el Gradient Boosting clásico quedaron un escalón por debajo, posiblemente porque no se optimizaron hiperparámetros más allá de la configuración base.
- **Modelos Lineales y Baseline**
Los métodos lineales (Regresión Lineal, Ridge y Lasso) obtuvieron un MAE promedio superior a 100, lo que indica que la relación entre las variables independientes y el precio no es puramente lineal.
- **SVR**
El SVR presentó un rendimiento intermedio, mejor que los lineales pero muy lejos de los árboles, sugiriendo que sus kernels no capturan tan eficazmente las

interacciones presentes.

5. Selección del Modelo Final

Con base en estos resultados, **Random Forest** es la opción óptima para avanzar al ajuste fino e implementación del modelo final. Sus ventajas son:

- **Precisión Superior:** MAE medio más bajo en validación.
- **Estabilidad:** Desviación estándar moderada, indicando consistencia entre pliegues.
- **Menor necesidad de escalado o transformación:** Funciona adecuadamente sobre datos no normalizados.
- **Interpretabilidad relativa:** Permite extraer importancias de variable para entender contribuciones al precio.

5. Entrenamiento Final: Ajuste de Hiper Parámetros para los Modelos Seleccionados

En esta fase se refinó el rendimiento de los seis modelos más prometedores identificados en la etapa de selección mediante búsqueda aleatoria de hiper parámetros (RandomizedSearchCV) con 60 iteraciones y validación cruzada de cinco pliegues. El objetivo fue optimizar cada algoritmo en función del MAE en validación, buscando configuraciones que mejoraran consistentemente su capacidad predictiva sin incurrir en sobreajuste.

5.1 Espacios de Búsqueda Definidos

Para cada modelo se estableció un rango de valores realistas y diversos que incluyó tanto parámetros de complejidad de estructura (profundidad máxima, número de árboles) como de regularización y muestreo (tasa de aprendizaje, fracción de características, tamaño mínimo de hojas y particiones, uso de bootstrapping).

- **RandomForest y ExtraTrees:** se exploraron entre 200 y 1 200 estimadores, profundidades de 10–60, fracciones de características entre 30 % y 100 %, y mínimos de muestra para división y hoja en rangos amplios.
- **GradientBoosting:** se consideraron tasas de aprendizaje de 0.01–0.20, árboles de profundidad 3–10, submuestreos de 60 %–100 % y configuraciones de nodo similares al ensamble anterior.
- **XGBoost:** se incluyeron además parámetros de regularización gamma, min_child_weight, y muestreo de columnas (colsample_bytree) para capturar su efecto

en el sesgo y la varianza del modelo.

5.2 Proceso de Optimización

Cada búsqueda aleatoria ajustó 300 combinaciones (60 iteraciones × 5 pliegues) y registró el mejor conjunto de parámetros según MAE negativo medio. Se utilizó `n_jobs=-1` para paralelizar el ajuste y acelerar el proceso. Tras completar el ajuste, se almacenaron los estimadores óptimos en archivos separados para preservar su configuración.

5.3 Resultados del Tuning

Los valores de MAE en validación cruzada y los parámetros mejor encontrados fueron:

- **RandomForest**
 - **MAE_CV:** 67.92
 - **Parámetros Clave:** ~812 árboles, profundidad máxima 31, fracción de características ≈92 %, sin bootstrap, muestras mínimas de división = 4, de hoja = 1.
- **ExtraTrees**
 - **MAE_CV:** 71.83
 - **Parámetros Clave:** configuraciones paralelas a RandomForest, reflejando la similar estructura de ensamble agresivo.
- **GradientBoosting**
 - **MAE_CV:** 72.26
 - **Parámetros Clave:** ~739 árboles, learning rate ≈0.15, profundidad = 9, subsample ≈83 %, muestras mínimas de división = 18, de hoja = 3.
- **XGBoost**
 - **MAE_CV:** 72.35
 - **Parámetros Clave:** ~738 árboles, learning rate ≈0.05, profundidad = 8, subsample ≈81 %, colsample_bytree ≈97 %, gamma ≈0.35, min_child_weight = 2.

Nota: Algunos intentos de ajuste de XGBoost fallaron debido a configuraciones demasiado extremas que recargaron la memoria GPU; estos casos se descartaron automáticamente.

5.4 Interpretación y Comparativa

- **Mejora sobre la versión base:** RandomForest mostró una ligera reducción del MAE, de 68.29 a 67.92, evidenciando que el ajuste fino de parámetros (profundidad y número de árboles) aportó una ganancia de precisión modesta pero estable.
- **Diferencial entre ensambles:** Aunque ExtraTrees y GradientBoosting obtuvieron resultados cercanos, RandomForest se mantuvo como el líder, probablemente por su robustez a variaciones en los datos y menor sensibilidad a hiperparámetros de muestreo.
- **Estabilidad vs. Complejidad:** XGBoost, si bien es más complejo de ajustar, alcanzó un MAE similar al GradientBoosting clásico, sugiriendo que para este problema concreto la ganancia de sus mecanismos de regularización avanzados fue limitada en comparación con el ensamble estándar.

6. Persistencia de los Modelos Afinados

Para consolidar los resultados del ajuste de hiperparámetros y garantizar la reproducibilidad del flujo de trabajo, se entrenaron los modelos finales con sus configuraciones óptimas utilizando la totalidad del conjunto de entrenamiento preprocesado. A continuación se describen los pasos y consideraciones de este proceso.

6.1 Definición de los Modelos con Parámetros Óptimos

Cada algoritmo se configuró con los valores hallados en la fase de tuning mediante búsqueda aleatoria. En particular:

- **RandomForestRegressor:**
 - 812 árboles
 - Profundidad máxima: 31
 - Fracción de características por árbol: ≈ 0.92
 - Muestras mínimas para división: 4
 - Muestras mínimas en hoja: 1
 - Sin bootstrap
- **ExtraTreesRegressor:**
 - Configuración paralela a RandomForest, replicando el número de estimadores, profundidad y muestreo de características, también sin bootstrap.
- **GradientBoostingRegressor:**
 - 739 estimadores
 - Tasa de aprendizaje: ≈ 0.152
 - Profundidad máxima: 9
 - Submuestreo de filas: ≈ 0.83

- Parámetros de nodo: división mínima = 18, hoja mínima = 3

- **XGBRegressor:**

- 738 iteraciones
- Learning rate: ≈ 0.05
- Profundidad: 8
- Submuestreo: ≈ 0.81
- Muestreo de columnas por árbol: ≈ 0.97
- Regularización gamma: ≈ 0.35
- min_child_weight = 2

- **LGBMRegressor:**

- Se empleó la configuración por defecto orientada a regresión con métrica MAE, manteniendo el mismo esquema de entrenamiento en paralelo (n_jobs=-1).

6.2 Entrenamiento Final

Cada modelo fue ajustado de manera independiente sobre el conjunto completo de entrenamiento (X_train_proc, y_train) para aprovechar toda la información disponible. Esta etapa asegura que el estimador incorpore el máximo de patrones estadísticos antes de la evaluación definitiva.

6.3 Guardado y Versionado

Tras el entrenamiento, cada modelo se serializó en disco usando joblib, generando un archivo .pkl con nombre representativo:

- random_forest_final.pkl
- extra_trees_final.pkl
- grad_boost_final.pkl
- xgboost_final.pkl
- lightgbm_final.pkl

7. Generación de la Submission

Para cerrar el flujo de trabajo y preparar la entrega en Kaggle, se aplican de forma idéntica los pasos de procesamiento al conjunto de prueba original y se generan las predicciones con el modelo afinado. A continuación se describen los hitos clave de este proceso.

7.1 Carga del Conjunto de Prueba Crudo

Se importa el archivo **Test.csv** sin aplicar ninguna transformación previa, garantizando que el pipeline completo pueda reproducirse exactamente desde los datos originales.

7.2 Reproducción del Feature Engineering

1. **Agrupación de ciudades raras:** Se identifican las categorías de city con menos del 1 % de presencia y se reemplazan por la etiqueta “Other”, igual que en entrenamiento.
2. **Nueva variable de distancia total:** Se calcula total_dist sumando dist y metro_dist para capturar la accesibilidad global.
3. **Interacción de capacidad y dormitorios:** Se genera cap_bedrooms multiplicando la capacidad máxima de personas por el número de dormitorios, reflejando la densidad de ocupación.

Estos pasos aseguran que las mismas características creadas en el conjunto de entrenamiento estén disponibles para el modelo en prueba.

7.3 Aplicación del Preprocesamiento

El ColumnTransformer entrenado sobre los datos de entrenamiento se reutiliza para:

- Imputar valores faltantes de las variables numéricas.
- Escalar y winsorizar los atributos continuos.
- Codificar las variables categóricas con one-hot encoding.

De este modo, **X_test_proc** queda alineado en dimensión y orden de columnas con **X_train_proc**, garantizando coherencia.

7.4 Predicción con el Modelo Afinado

Se carga el archivo serializado (random_forest_final.pkl) que contiene el **Random Forest** ajustado con los mejores hiperparámetros. Al invocar su método predict sobre **X_test_proc**, se obtienen las estimaciones de realSum.

7.5 Creación del Archivo

Finalmente, se construye un DataFrame que asocia cada id de **Test.csv** con la predicción correspondiente de realSum. Usando el método to_csv, se exporta un archivo **submission.csv** sin índice, con exactamente 15 513 filas, listo para ser subido a la plataforma de evaluación.

8. Mezcla de Modelos (“Stacking”)

En esta etapa se combina la fuerza de distintos algoritmos base para generar un metamodelo que aproveche sus ventajas complementarias y reduzca el error de predicción. El procedimiento se articula en las siguientes fases:

8.1 Preparación de Datos y Definición de Base Learners

Se reutiliza el conjunto de entrenamiento preprocesado ($X_{\text{train_proc}}$, y_{train}) y se definen tres modelos base con sus hiperparámetros optimizados previamente:

- **Random Forest:** ensamble robusto capaz de capturar relaciones no lineales y gran variedad de interacciones.
- **XGBoost:** boosting especializado con mecanismos de regularización avanzados para ajustar sesgo y varianza.
- **LightGBM:** variante de boosting optimizado para eficiencia en gran volumen de datos.

8.2 Generación de Predicciones Out-Of-Fold (OOF)

Para cada modelo base se realiza validación cruzada de 5 pliegues y, en cada partición, se entrena con cuatro quintas partes de los datos y se predice sobre el quinto restante. De este modo se obtienen predicciones “fuera de muestra” para todas las instancias del entrenamiento, garantizando que los valores de entrada al metamodelo no hayan sido vistos durante el entrenamiento de cada base learner. Se calcula el MAE OOF de cada modelo como indicador de su desempeño individual.

8.3 Entrenamiento de los Base Learners en Todo el Conjunto

Tras recopilar las predicciones OOF, cada base learner se reentrena con la configuración óptima sobre la totalidad del conjunto de entrenamiento, asegurando que incorporen toda la información disponible antes de la fase de combinación.

8.4 Construcción del Meta-Learner

Las predicciones OOF de los tres modelos forman un nuevo conjunto de características (una columna por modelo base). Con ellas se entrena un regresor lineal de Ridge, el cual aprende a

combinar las salidas de los modelos base asignando pesos que minimicen el MAE global. El error OOF del meta-learner sobre estas predicciones se evalúa para comprobar la mejora respecto a los modelos individuales.

8.5 Generación de Predicciones Finales

Para producir la entrega definitiva:

1. Se aplica cada modelo base (ya entrenado con todo el conjunto) al conjunto de prueba procesado, obteniendo tres vectores de predicciones.
2. Se ensamblan estas predicciones en un nuevo array de características de test.
3. El meta-learner predice el valor final de realSum a partir de esa matriz, integrando las fortalezas de cada algoritmo base.
4. Se guarda el archivo `submission_stackingv2.csv` con los IDs originales y las predicciones finales, listo para su evaluación en Kaggle.